ST7 : Efficacité des systèmes d'énergie embarqués

Rapport de projet 1

Optimisation de la composition d'un micro-réseau pour des Data Centers

Réalisé par :

Beatriz BRIOLI DOLL DE MORAES

Martin SCHULZ SAIP

Mathilde VERBRIGGHE

CentraleSupélec – Gif-sur-Yvette

30 mars 2026

Table des matières

1 Introduction **2**

2 Étude des outils de simulation et d'optimisation **2**
2.1 Familiarisation avec le simulateur Vessim 2
2.2 Prise en main de la bibliothèque Pymoo 3

3 Analyse paramétrique et comportement du système **3**
3.1 Cartographie des variables 4
3.2 Simulations et Observations 5
3.2.1 Analyse des scénarios limites 5
3.2.2 Politique de coût 6
3.2.3 Analyse de la dégradation et cycles de vie 7
3.3 L'algorithme Rainflow 8

4 Implémentation de l'Optimisation **9**
4.1 Optimisation Mono-objectif 9
4.1.1 Variables de décision 9
4.2 Optimisation Multi-objectif 10
4.2.1 Dégradation dynamique de la batterie (algorithme Rainflow) 11
4.2.2 Évolution du Front de Pareto 12
4.2.3 Étude Comparative entre la France et le Royaume-Uni 12

5 Conclusion **13**

Références **14**

1 Introduction

L'expansion rapide des services numériques et l'émergence massive de l'intelligence artificielle générative ont entraîné une demande en énergie sans précédent des centres de données. Cette croissance exponentielle se traduit par une augmentation significative de la consommation électrique : entre 2017 et 2021, la consommation des principaux acteurs du secteur a plus que doublé, et les prévisions de l'Agence Internationale de l'Énergie suggèrent un nouveau doublement d'ici 2030.

Cette situation place les réseaux électriques sous une tension critique, créant des goulots d'étranglement qui freinent désormais le développement technologique. Face à l'incapacité des infrastructures à suivre ce rythme, l'intégration de micro-réseaux locaux, combinant des sources renouvelables et du stockage, est devenue une solution stratégique pour en garantir l'autonomie et la durabilité.

Ce projet a pour objectif central le développement et l'implémentation d'algorithmes d'optimisation appliqués au dimensionnement de ces micro-réseaux. L'importance de cette étude réside dans la complexité de l'équilibre entre des facteurs souvent conflictuels : la performance économique et l'impact environnemental. Pour ce faire, nous utilisons le simulateur *Vessim* [2], qui permet une co-simulation précise entre la charge informatique et le système d'énergie, intégré à la bibliothèque *Pymoo* [3] pour la résolution des problèmes mathématiques d'optimisation.

Notre démarche s'inspire de la méthodologie récemment proposée par Irion *et al.* [1]. Leurs travaux présentent un framework d'optimisation utilisant Vessim pour explorer différentes compositions de micro-réseaux et analyser les compromis directs entre les émissions de carbone opérationnelles et embarquées.

Le travail est structuré autour de deux approches principales :

— **Optimisation mono-objectif :** Axée strictement sur la viabilité économique, elle vise à trouver la configuration présentant le coût financier le plus bas pour l'exploitation du système, avec une contrainte sur les émissions de CO2 générées.

— **Optimisation multi-objectif :** Nous y explorons le conflit entre la minimisation du coût et la réduction des émissions de CO_2 tout au long du cycle de vie des éléments.

À travers ces implémentations en Python, nous cherchons non seulement à identifier le meilleur scénario pour l'étude de cas proposée, mais aussi à consolider notre maîtrise pratique des outils modernes de simulation et des algorithmes d'optimisation appliqués à l'efficacité énergétique.

2 Étude des outils de simulation et d'optimisation

2.1 Familiarisation avec le simulateur Vessim

Le simulateur *Vessim* [2], développé par l'Université technique de Berlin, constitue un élément fondamental de notre démarche de modélisation. Il s'agit d'un banc d'essai de co-simulation destiné aux systèmes tenant compte de leur empreinte carbone, qui permet de simuler l'interaction des systèmes informatiques avec les sources d'énergie renouvelables locales, les systèmes de stockage par batterie et le réseau public. Cela nous permet alors d'analyser le comportement des centres de données équipés de micro-réseaux.

L'architecture de Vessim repose sur le cadre de co-simulation *Mosaik*, qui permet de réutiliser et de combiner des modèles et simulateurs existants pour générer des scénarios de réseaux intelligents à grande échelle, et s'organise autour de plusieurs fichiers que nous avons manipulés (cf. partie 3.1 pour la description complète) :

— **vs.Environment** : gère l'horloge système et définit la résolution temporelle de la simulation, fixée à 5 minutes (step_size = 300) pour nos premiers essais.
— **vs.Actor** : représente les entités physiques consommatrices, comme le serveur principal avec une charge constante, ou productrices comme les panneaux solaires et les éoliennes.
— **vs.SimpleBattery** : modélise le système de stockage avec sa capacité physique et ses limites de sécurité, telles que le *Min SoC* (State of Charge) fixé à 20% pour éviter les dégradations profondes.
— **vs.Policy** : contient la logique de contrôle, telle que la *SmartCarbonPolicy*, qui décide de la répartition de l'énergie en fonction des signaux d'intensité carbone du réseau public.
— **vs.Microgrid** et **vs.MemoryLogger** : respectivement le fichier regroupant les acteurs et l'outil d'enregistrement de l'historique des données pour l'analyse.

Un atout majeur de cet outil est son intégration avec le *System Advisor Model* (SAM) du NREL via le package *PySAM* [4]. Cette extension nous a permis d'utiliser des modèles pour convertir des données météorologiques réelles en profils de puissance électrique précis.

2.2 Prise en main de la bibliothèque Pymoo

Pour résoudre les problèmes d'optimisation identifiés, nous avons utilisé *Pymoo*, une bibliothèque Python spécialisée dans l'optimisation multi-objectif. Son approche orientée objet facilite la définition de problèmes complexes en héritant de classes telles que *ElementwiseProblem*.

Notre apprentissage de Pymoo s'est déroulé en plusieurs phases :
— **Validation sur des fonctions tests** : nous avons testé la résolution de problèmes d'optimisation théoriques connus, comme la fonction de *Rastrigin* pour l'optimisation mono-objectif et le problème *TNK* pour l'optimisation multi-objectif, afin de valider la convergence des algorithmes malgré des contraintes non linéaires.
— **Utilisation de l'algorithme NSGA-II** : Pour notre cas réel, nous avons privilégié l'algorithme génétique *Non-dominated Sorting Genetic Algorithm II* (NSGA-II). Cet algorithme est particulièrement efficace pour explorer l'espace des paramètres et identifier le front de Pareto, représentant les compromis optimaux entre les émissions opérationnelles et les émissions incorporées de l'infrastructure dans notre cas.
— **Méthodologie "Black-box"** : En raison de la complexité des interactions simulées dans Vessim, la fonction objectif est évaluée directement par le simulateur. Pymoo traite donc le système comme une "boîte noire" (*black-box*) , optimisant les variables de décision (puissance en énergie éolienne, puissance solaire et capacité de batterie) sans nécessiter de forme analytique simple.

3 Analyse paramétrique et comportement du système

Cette section constitue une étape importante de l'étude, assurant la transition entre la prise en main technique des outils et la phase d'optimisation proprement dite. La conception d'un système énergétique efficace nécessite une compréhension fine de la sensibilité du micro-réseau face aux différentes variables de conception et aux contraintes. L'approche adoptée s'articule donc autour de deux axes principaux. Dans un premier temps, nous avons établi une description de l'architecture logicielle du simulateur afin d'identifier les variables critiques et les leviers d'action disponibles. Dans un second temps, le système est soumis à une série de simulations et de scénarios limites où nous faisons varier les différentes valeurs des comppsants. Cette analyse de comportement permet de valider la cohérence physique des modèles, d'observer l'impact des

politiques de gestion sur la facture électrique et d'évaluer la durabilité des infrastructures de stockage avant la formulation du problème d'optimisation multi-objectif.

3.1 Cartographie des variables

L'analyse de l'architecture de *Vessim* permet de comprendre l'interaction entre les différents modules. Le tableau suivant (1) synthétise les fonctions de chaque fichier ainsi que les variables clés identifiées pour notre étude.

Fichier	**Fonction**	**Variables et Leviers d'action**
Init	Point d'entrée et organisateur de l'architecture.	Aucune variable de réglage direct.
Broker	Communication API entre la simulation et l'extérieur.	`commands`, `port`, `export_prometheus`, `storage_prop`, `policy_prop`, `actor_signal`, `microgrid_battery_soc`, `microgrid_battery_capacity_wh`.
Data	Gestion des datasets externes (CSV météo/carbone).	`dataset`, `scale`, `start_time`, `use_forecast`, `actual`, `forecast`, `fill_method`.
Util	Fonctions utilitaires, temps et logs.	`sim_start`, `behind_threshold`, `simtime`, `dt`.
Actor	Définition des producteurs et consommateurs.	`name`, `signal`, `step_size`, `power`, `state`.
Controller	Moniteur, décideur et sauvegarde des données.	`controllers`, `export_prometheus`, `target`, `prop`, `val`, `microgrid_states`, `commands`.
Environnement	Gestion de la co-simulation (Mosaik).	`sim_start`, `step_size`, `until`, `rt_factor`, `behind_threshold`.
Microgrid	Système physique central et bilan de puissance.	`actors`, `policy`, `storage`, `grid_signals`, `p_delta`, `p_grid`.
Plot	Outils de visualisation interactive.	`trace`, `scale`, `df`, `p_delta`, `storage_state.soc`.
Policy	Algorithme décisionnel (gestion du surplus/manque).	`mode`, `charge_power`, `energy_delta`.
Signal	Interface de données statiques ou temps réel.	`value`, `forecast`, `scale`, `update_interval`, `query`, `WatttimeSignal`.
Storage	Simulation physique des batteries.	`capacity`, `initial_soc`, `min_soc`, `c_rate`, `number_of_cells`.

TABLE 1 – Cartographie des fichiers, fonctions et variables du simulateur Vessim (voir version complète en annexe)

Ainsi *Vessim* n'est pas un simple bloc mais un écosystème où chaque fichier joue un rôle précis dans la chaîne de valeur énergétique. Pour notre projet d'optimisation, nous avons identifié que les leviers les plus puissants se situent dans les fichiers **Data/Signal** (via le `scale` pour dimensionner le champ solaire), **Storage** (via le `capacity` et le `number_of_cells` pour le stockage) et **Policy** (via le `charge_power` et le `mode`).

La variable `p_delta` dans le fichier **Microgrid** est l'indicateur de performance le plus critique : elle est le résultat direct de nos choix de dimensionnement. Un `p_delta` trop souvent

négatif indique un système sous-dimensionné dépendant du réseau, tandis qu'un surplus mal géré indique un manque de capacité de stockage ou une politique de charge inefficace. Cette compréhension fine des variables nous permet désormais de lier les paramètres d'entrée (*inputs*) aux objectifs de l'optimisation multi-objectif (Coût vs CO_2).

3.2 Simulations et Observations

Avant de lancer les algorithmes d'optimisation globaux, nous avons développé plusieurs scripts de test pour évaluer la sensibilité du système face à des configurations extrêmes. Ces essais nous ont permis de valider la dynamique entre la charge informatique (fixée à 1500W), la production intermittente et la réponse du stockage.

3.2.1 Analyse des scénarios limites

Quatre scénarios principaux ont été modélisés pour établir des points de comparaison (*baselines*) :

- **100% Grid :** ce script simule un Data Center sans aucune infrastructure de micro-réseau. Le système achète 1500W en continu sur le réseau public, représentant le coût et l'empreinte carbone maximaux.
- **Production sans stockage (0% Batterie) :** Nous avons testé l'impact de l'installation de panneaux solaires (`scale: 4000`) et d'éoliennes sans batterie. L'observation majeure est le déphasage : le surplus d'énergie est exporté inutilement le jour, tandis que la dépendance au réseau reste totale la nuit. Un essai spécifique **"100% Soleil"** avec un surdimensionnement solaire (`scale: 8000`) a montré que même avec une production doublée, l'absence de batterie empêche toute autonomie nocturne.
- **Mode Iloté (100% Off-grid) :** À l'inverse, nous avons testé la viabilité d'un système totalement déconnecté du réseau (`mode: "islanded"`). Pour éviter un "blackout" du Data Center, ce scénario nécessite une batterie très importante (25kWh), donc chère, et une production solaire doublée (`scale: 8000`). Ce test a prouvé que l'autonomie est techniquement possible mais économiquement et environnementalement coûteuse (émissions incorporées élevées).

FIGURE 1 – Scénario 100% Grid (Baseline).

FIGURE 2 – Production renouvelable sans stockage.

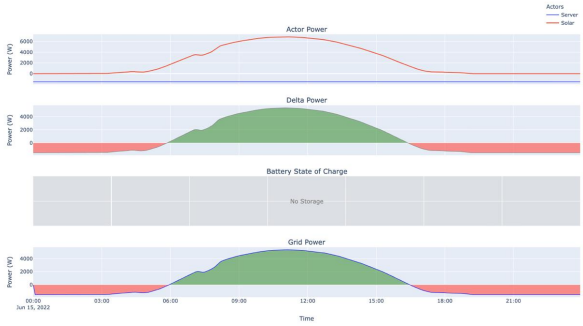


FIGURE 3 – Scénario 100% Solaire (surdimensionné).

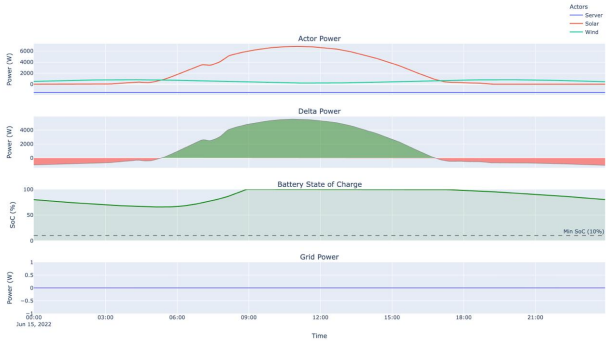


FIGURE 4 – Mode Iloté (100% Off-grid).

FIGURE 5 – Visualisation des quatre scénarios limites de la section 3.2.1.

3.2.2 Politique de coût

Nous avons implémenté une classe personnalisée, la `HeuresCreusesPolicy`, pour simuler une stratégie de gestion axée uniquement sur la réduction de la facture électrique.

La batterie est forcée à se charger à puissance maximale (2500W) durant les heures creuses (entre 22h et 06h), moment où l'électricité est la moins chère. Durant les heures pleines, le système privilégie la décharge de la batterie pour couvrir les 1500W du serveur, minimisant ainsi l'achat au tarif fort. Ce script démontre l'importance de la batterie comme outil d'arbitrage économique.

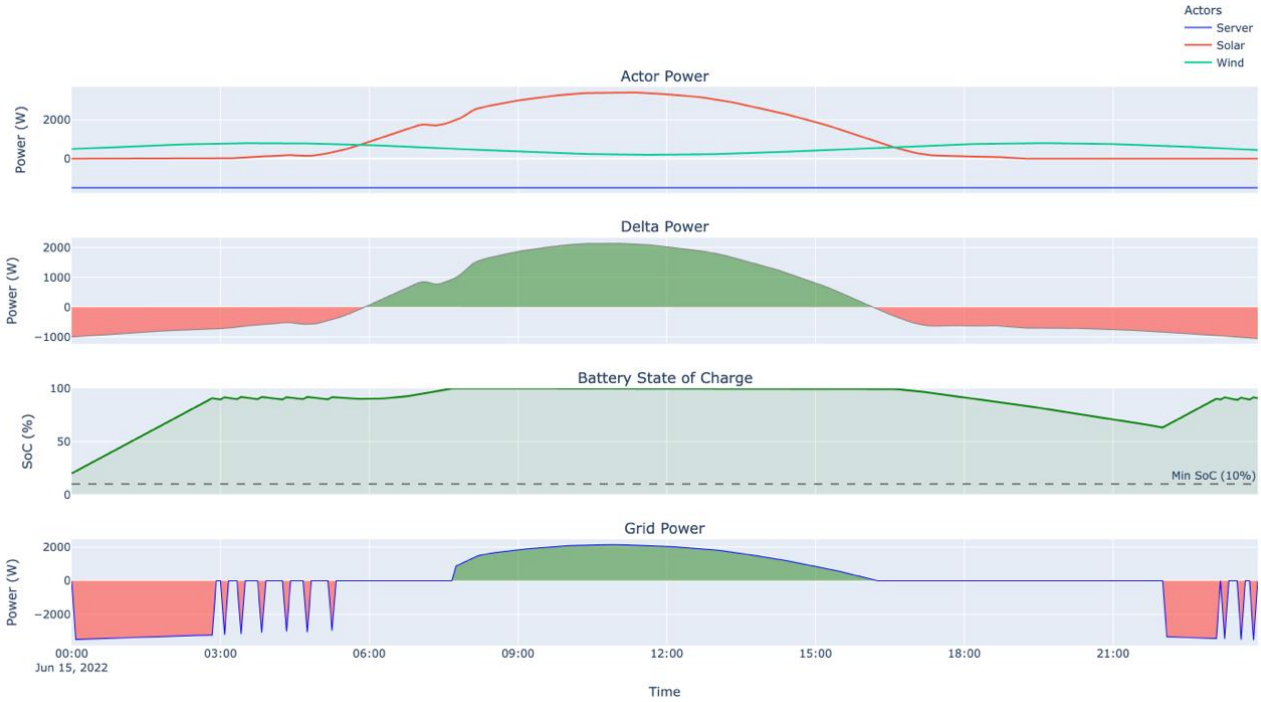


FIGURE 6 – Impact de la politique tarifaire sur le comportement de la batterie e le p_delta.

3.2.3 Analyse de la dégradation et cycles de vie

Un aspect crucial de nos simulations a été l'évaluation de la durabilité du matériel. Nous avons créé un script de simulation sur 7 jours pour calculer l'usure de la batterie.

Le code calcule la somme des variations absolues de l'état de charge (SoC) pour déterminer le nombre de "cycles équivalents". Pour une batterie de 10kWh avec une décharge minimale (`min_soc`) de 20%, nous avons projeté une durée de vie de 11,6 ans sur la base de 292 cycles par an.

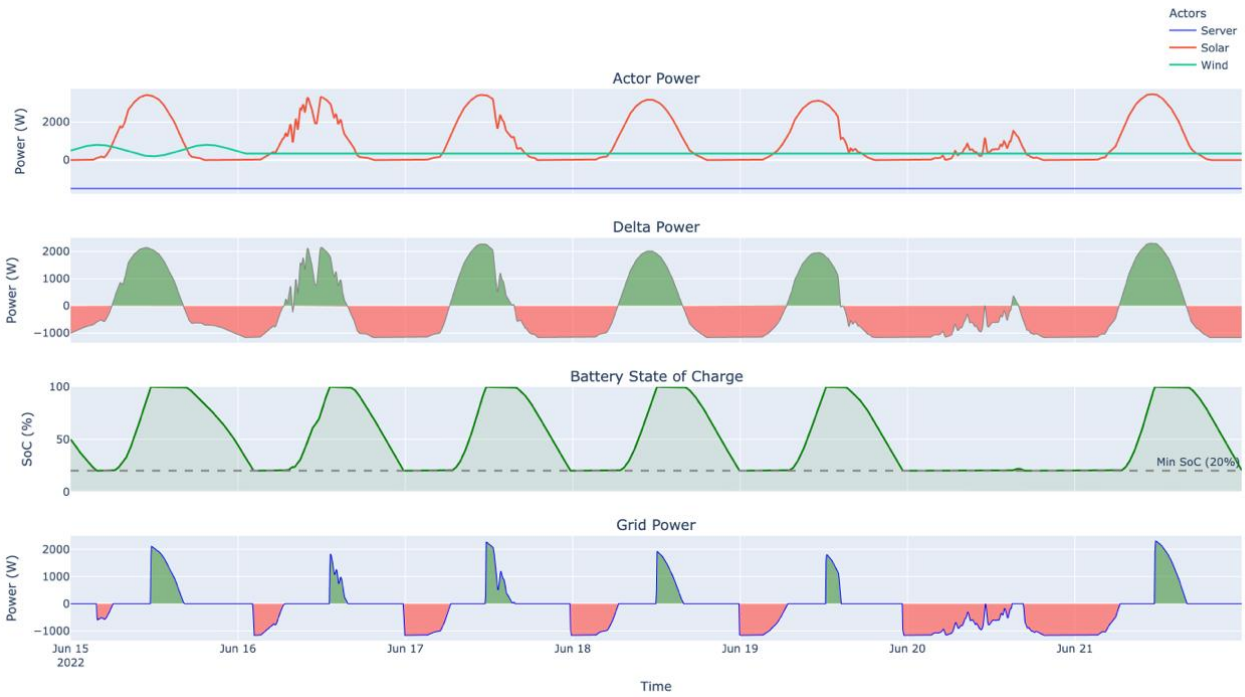


FIGURE 7 – Évolution du SoC sur 7 jours et calcul de la dégradation cyclique.

3.3 L'algorithme Rainflow

La durabilité du système de stockage est un facteur critique tant sur le plan économique (amortissement du CAPEX) que sur le plan environnemental. Pour évaluer précisément l'usure de la batterie, nous avons comparé deux méthodologies de calcul du nombre de cycles de charge.

Initialement, une méthode "commune" a été testée, consistant à additionner toutes les variations de l'état de charge (SoC) et à diviser le total par deux pour obtenir un nombre de cycles équivalents. Cependant, cette approche présente un défaut majeur : elle traite chaque petite oscillation (causée par exemple par un passage nuageux ou une rafale de vent) comme une fraction d'un grand cycle, ignorant ainsi la structure réelle des « cycles imbriqués ». Pour des profils d'énergies renouvelables où le bruit sur le SoC est constant, cette méthode manque de précision chirurgicale.

Pour pallier ces limites, nous avons implémenté l'algorithme Rainflow, une méthode issue de l'ingénierie des matériaux pour analyser la fatigue mécanique. Son fonctionnement repose sur une analogie physique :

- Le graphique du SoC est pivoté de 90°, transformant le profil temporel en une série de "toits" successifs.
- L'algorithme simule des gouttes d'eau s'écoulant depuis chaque pic et chaque creux.
- Le flux s'arrête lorsqu'il rencontre un pic plus haut ou un creux plus profond que son origine.
- La distance horizontale parcourue définit la Profondeur de Décharge (DoD - *Depth of Discharge*) réelle de chaque cycle.

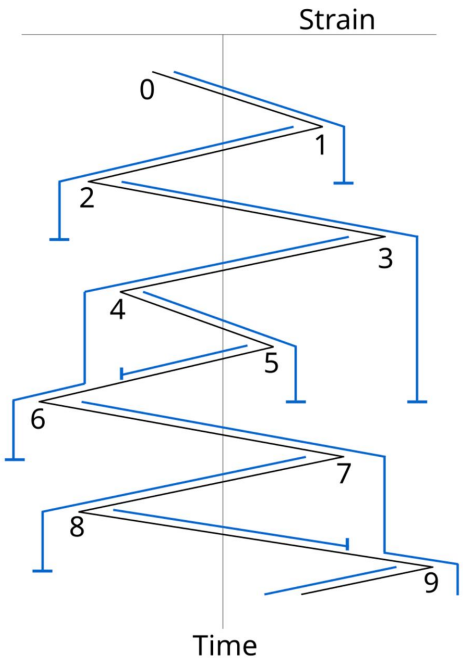


FIGURE 8 – Analogie du ruissellement d'eau pour l'identification des cycles (Algorithme Rainflow).

L'avantage majeur de Rainflow est sa capacité à identifier les réelles inversions de tendance. Il distingue parfaitement un cycle complet (décharge profonde de 100% à 20%) d'une série de micro-oscillations (8 cycles entre 100% et 90%), ces deux scénarios n'ayant pas le même impact sur la chimie de la batterie.

Dans nos premières simulations avec des profils solaires standards, les résultats des deux méthodes de calcul de cycles avaient tendance à converger en raison de la régularité des charges et décharges. Afin de ne pas gaspiller le potentiel analytique de l'algorithme Rainflow et de

tester la robustesse du système dans des conditions réelles, nous avons délibérément modifié les données d'entrée du code.

Nous avons sélectionné un jeu de données météorologiques présentant une instabilité nuageuse (variations rapides de l'irradiance). Ce changement a introduit un « bruit » significatif sur l'état de charge (SoC) de la batterie. De plus, pour minimiser le stress sur les cellules et prolonger la durée de vie du système, nous avons imposé une contrainte de fonctionnement stricte, limitant la charge maximale à 80% ($max_soc = 0,8$).

4 Implémentation de l'Optimisation

4.1 Optimisation Mono-objectif

Dans cette phase, l'objectif est de trouver la configuration technique la plus économique pour le micro-réseau, tout en respectant une contrainte environnementale fixe.

Pour garantir la précision de la simulation, un travail important de collecte de données pour l'année 2022 a été réalisé. Les sources suivantes ont été fusionnées dans um fichier unique (*vessim_unified_data_2022.csv*) avec une résolution horaire :

- **Données Réseau (RTE) :** Historique de la consommation nationale et, surtout, l'intensité carbone du mix électrique français en temps réel.
- **Données de Marché (Energy-Charts) :** Prix spot de l'électricité (€/MWh) pour l'année 2022.
- **Profils Renouvelables (PySAM) :** Génération théorique pour 1 kW installé de solaire et d'éolien, calculée spécifiquement pour les coordonnées géographiques de Gif-sur-Yvette.

Ces données permettent de modéliser les variations saisonnières de production. Comme illustré par les profils de production (Figure 9), on observe une production solaire forte en été et une production éolienne plus robuste durant les mois d'hiver.

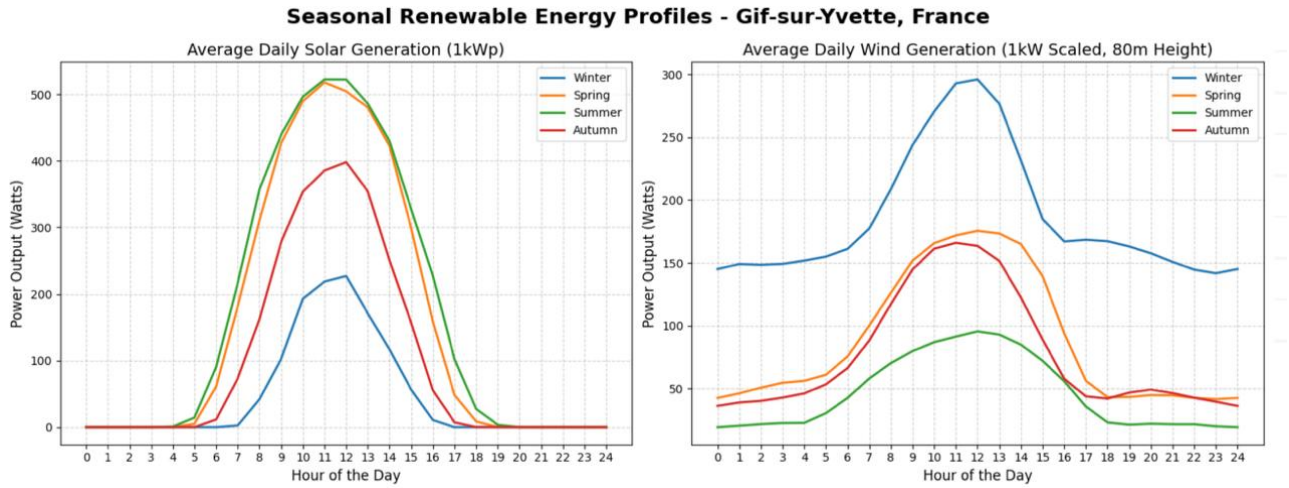


FIGURE 9 – Représentation des profils saisonniers de production renouvelable à Gif-sur-Yvette.

Le problème est formulé mathématiquement pour être résolu par l'algorithme génétique NSGA-II via la bibliothèque *Pymoo*.

4.1.1 Variables de décision

Le système doit optimiser trois paramètres (vecteur x) :

— x_1 : Puissance éolienne installée (Borne : [0, 5000] kW)
— x_2 : Puissance solaire installée (Borne : [0, 3000] kWp)
— x_3 : Capacité de stockage par batterie (Borne : [200, 10000] kWh)

L'objectif unique est la minimisation du coût opérationnel total (OPEX) sur un an :

$$\min f(x) = \sum_{t=1}^{8760} (\text{Prix}_t \times \text{Achat_Réseau}_t) \tag{1}$$

Une contrainte d'intensité carbone moyenne a été fixée à **79 gCO2e/kWh**, ce qui correspond au facteur carbone français actuel (DPE). L'algorithme rejette toute configuration dont l'empreinte carbone moyenne sur l'année dépasse ce seuil :

$$g(x) = \text{Moyenne}(\text{Intensité_Carbone}_{\text{consommée}}) - 79 \leq 0 \tag{2}$$

Après l'exécution du processus d'optimisation, la configuration optimale trouvée pour répondre à la demande d'un Data Center consommant 1 MW (taille moyenne) est la suivante :

Composant	**Valeur Optimale**
Puissance Éolienne	3 624 kW
Puissance Solaire	2 626 kWp
Capacité de Batterie	8 724 kWh
Coût Total Annuel	**1 359 199,31 €**

Table 2 – Résultats de l'optimisation mono-objectif.

On remarque que l'algorithme a privilégié une capacité de batterie importante (proche de la borne supérieure) pour maximiser l'utilisation des énergies renouvelables locales et éviter l'achat d'électricité sur le réseau lorsque l'intensité carbone dépasse le seuil autorisé. L'équilibre entre éolien et solaire reflète la nécessité de couvrir la charge de 1 MW de manière constante tout au long des différentes saisons de l'année 2022 en France. Cependant, on ne prend pas en compte ici les émissions et les coûts liés à l'installation, la fabrication ou le transport des composants.

4.2 Optimisation Multi-objectif

L'optimisation mono-objectif a démontré qu'il était possible de dimensionner un système économiquement viable tout en respectant une contrainte carbone fixe. Cependant, les émissions embarquées liées à la fabrication des équipements (CAPEX CO_2), et les émissions opérationelles (OPEX CO_2) sont souvent en conflit direct. Pour minimiser l'utilisation du réseau (OPEX), il faut surdimensionner les infrastructures locales (panneaux, éoliennes, batteries), ce qui augmente le bilan carbone de la fabrication (CAPEX) et le coût financier.

Pour analyser ce compromis, nous sommes passés à une optimisation multi-objectif sans contrainte stricte, en cherchant à minimiser trois objectifs en même temps pour notre vecteur de variables de décision $x = (x_{\text{éolien}}, x_{\text{solaire}}, x_{\text{batterie}})$:

1. **OPEX CO_2 (f_1)** : Émissions liées à l'achat d'électricité sur le réseau public (en kg CO_2). L'algorithme somme les émissions heure par heure sur l'année complète :

$$f_1(x) = \sum_{t=1}^{8760} (\text{Achat_Réseau}_t(x) \times \text{Intensité_Carbone}_t)$$

2. **CAPEX CO_2 (f_2)** : Émissions annualisées standards liées à la fabrication et l'installation des équipements locaux [8]. Les panneaux solaires et les éoliennes ont une durée de vie fixe estimée à 20 ans, tandis que la durée de vie de la batterie est calculée dynamiquement par l'algorithme Rainflow selon son utilisation :

$$f_2(x) = \frac{x_{\text{éolien}} \times 200}{20} + \frac{x_{\text{solaire}} \times 600}{20} + \frac{x_{\text{batterie}} \times 100}{\text{Durée_Vie_Batterie}(x)}$$

3. **Coût Total Annuel (f_3)** : Somme de la facture électrique (OPEX), de l'amortissement du matériel (CAPEX financier) et d'une taxe carbone simulée à 70 €/tonne [9] appliquée sur les opérations :

$$f_3(x) = \text{Coût_Réseau}(x) + \text{CAPEX_Financier}(x) + \left(f_1(x) \times \frac{70}{1000} \right)$$

Avec l'amortissement financier calculé similairement au CAPEX carbone :

$$\text{CAPEX_Financier}(x) = \frac{x_{\text{éolien}} \times 1700}{20} + \frac{x_{\text{solaire}} \times 900}{20} + \frac{x_{\text{batterie}} \times 450}{\text{Durée_Vie_Batterie}(x)}$$

4.2.1 Dégradation dynamique de la batterie (algorithme Rainflow)

Afin d'avoir un calcul plus realiste du CAPEX financier et environnemental de la batterie, nous avons couplé notre fonction d'évaluation à l'algorithme *Rainflow* (détaillé en section 3.3).

Au lieu de supposer une durée de vie fixe de 5 ou 10 ans pour la batterie, l'algorithme analyse le profil de charge/décharge (SoC) simulé par *Vessim* sur l'année entière. En appliquant la règle de Palmgren-Miner simplifiée pour les batteries Lithium-Ion, le code calcule les dommages cumulés causés par les profondeurs de décharge (DoD) de chaque micro-cycle. L'algorithme de *Pymoo* est ainsi contraint de trouver des solutions qui ne "maltraiteraient" pas la batterie, car une usure prématurée forcerait son remplacement, pénalisant lourdement les trois objectifs simultanément.

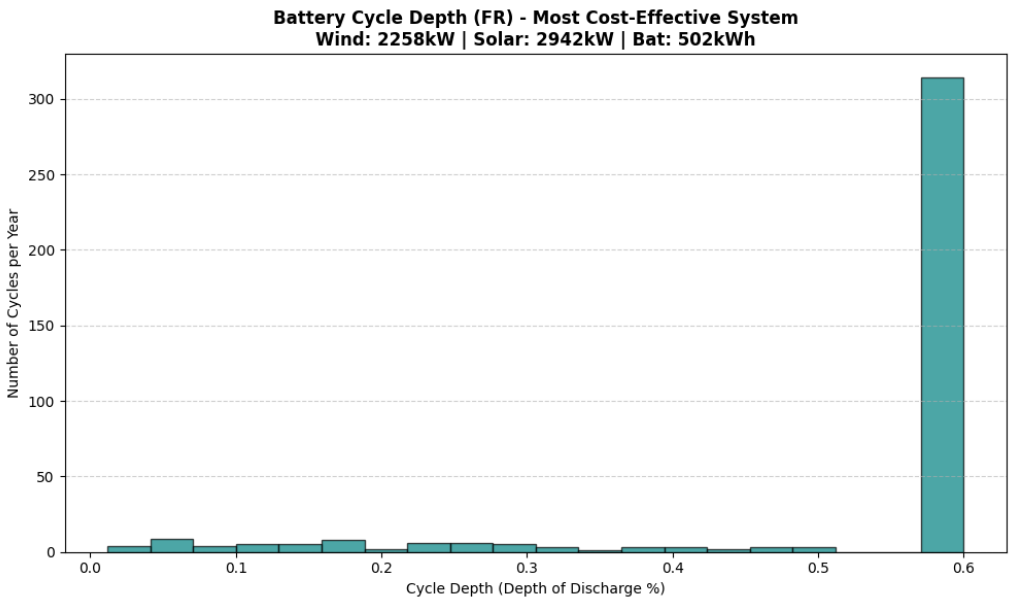


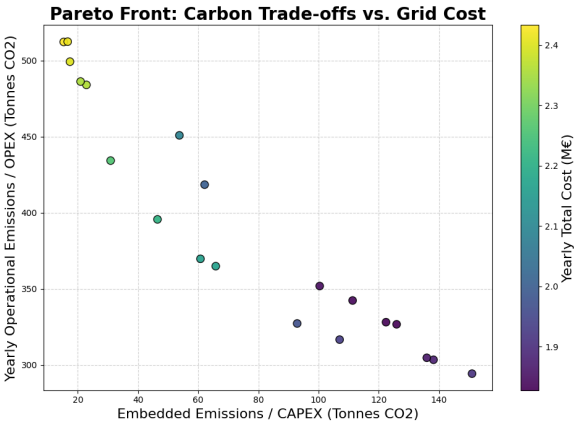
FIGURE 10 – Distribution des profondeurs de cycles (DoD) de la batterie via la méthode Rainflow pour la solution la plus rentable.

4.2.2 Évolution du Front de Pareto

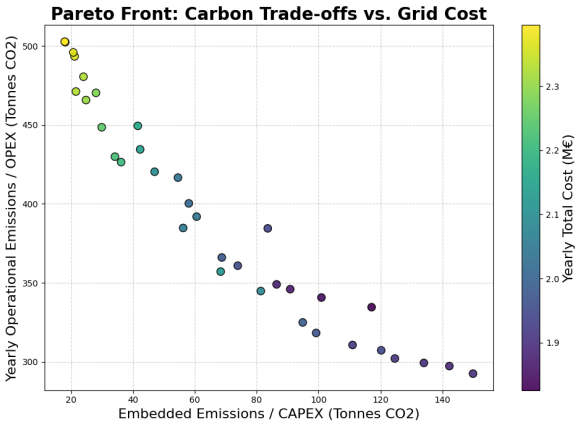
Afin de valider l'utilisation de l'algorithme NSGA-II et d'observer son comportement d'apprentissage, nous avons commencé par étudier l'évolution de la convergence sur notre scénario de référence (Gif-sur-Yvette).

Le budget de calcul, défini par le produit de la taille de la population (pop_size) par le nombre de générations (n_gen), dicte l'équilibre entre l'exploration de nouvelles configurations et l'exploitation pour affiner les meilleures solutions. La Figure 11 illustre cette progression analytique :

- **À 200 simulations** Avec $pop_size = 20$ et $n_gen = 10$, l'algorithme commence à dessiner une tendance globale, mais le front de Pareto est encore très discontinu. Les solutions sont dispersés et encore éloignées de l'optimum mathématique absolu.
- **À 700 simulations** Avec $pop_size = 35$ et $n_gen = 20$, l'espace de recherche est beaucoup mieux couvert. Le front de Pareto se densifie et commence à paraître continu. On peut voir des nouvelles solutions plus optimales qui minimisent conjointement les coûts et les émissions.



Front de Pareto (200 simulations)



Front de Pareto (700 simulations)

FIGURE 11 – Évolution de la convergence de l'algorithme NSGA-II pour Gif-sur-Yvette.

Cette étape de calibration a permis de justifier le lancement de l'expérimentation finale à **1500 simulations** ($pop_size = 50$ et $n_gen = 30$), garantissant ainsi un front de Pareto assez continu et une convergence optimale avant de réaliser notre analyse comparative internationale.

4.2.3 Étude Comparative entre la France et le Royaume-Uni

Pour tester la robustesse de notre modèle d'optimisation, nous avons fait une étude comparative entre deux réalités énergétiques et météorologiques différentes pour l'année 2022 :

- **Gif-sur-Yvette (France)** : Un réseau électrique fortement décarboné (base nucléaire, $\sim$60 gCO_2/kWh) mais un gisement éolien local modéré.
- **Londres (Royaume-Uni)** : Un réseau électrique fortement carboné (dépendant du gaz, $\sim$200-300 gCO_2/kWh) avec des prix très élevés suite à la crise énergétique, mais bénéficiant d'un gisement éolien important.

Nous avons généré les profils météorologiques spécifiques à Londres via PySAM (fichiers TMY du NREL) et intégré les données d'intensité carbone de l'opérateur britannique NESO [6]. L'algorithme NSGA-II a ensuite été exécuté pour les deux scénarios (Population : 50, Générations : 30, soit 1500 simulations par pays).

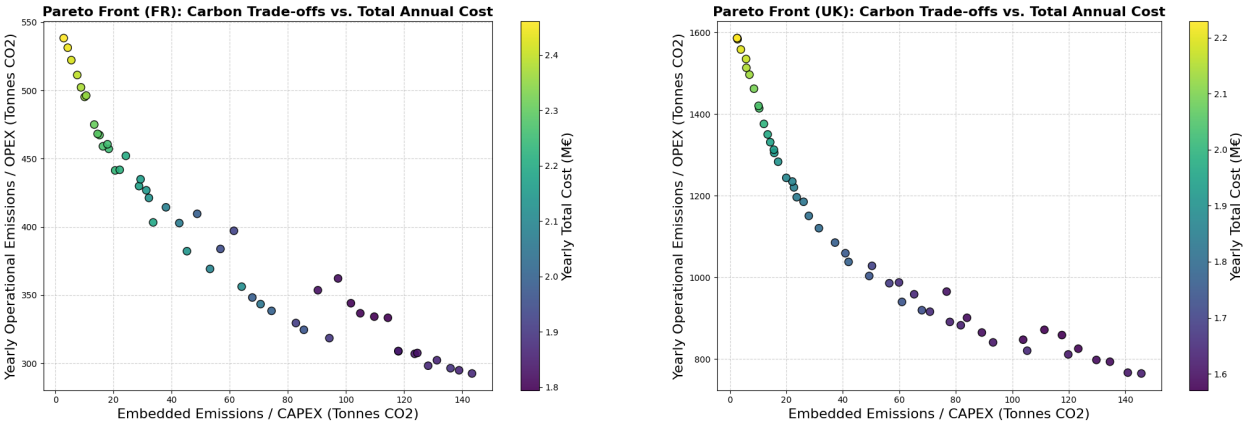


FIGURE 12 – Fronts de Pareto obtenus pour Gif-sur-Yvette (gauche) et Londres (droite).

Analyse des Résultats :
Les resultats de l'algorithme génétique observées dans la Figure 12 donnent des stratégies d'investissement qui changent avec la géographie :

1. **Le bilan carbone (L'avantage Française) :** À Gif-sur-Yvette, l'OPEX CO_2 annuel oscille entre 300 et 550 tonnes. À Londres, malgré des investissements matériels massifs, l'OPEX CO_2 explose entre 800 et 1600 tonnes. Dès que le vent tombe, le Data Center britannique tire sur un réseau fonctionnant au gaz, ce qui rend impossible l'atteinte des standards écologiques français, peu importe la taille de la batterie.
2. **Le bilan financier (L'avantage Britannique) :** De manière contre-intuitive, le coût total annuel s'avère moins cher à Londres (1.6 M€ à 2.2 M€) qu'à Gif-sur-Yvette (1.8 M€ à 2.45 M€). L'algorithme a compris que pour fuir un réseau anglais très cher, il suffisait de capitaliser sur le vent londonien en surdimensionnant les éoliennes. L'énergie générée gratuitement efface le coût électrique elevée, rendant le système très rentable économiquement, mais désastreux écologiquement.

5 Conclusion

Ce projet a permis la modélisation, l'analyse et l'optimisation d'un micro-réseau renouvelable dédié à l'alimentation d'un Data Center de 1 MW. En combinant l'environnement de co-simulation *Vessim* et les algorithmes génétiques de *Pymoo*, nous avons pu réaliser une Analyse du Cycle de Vie (ACV).

L'intégration d'algorithmes physiques précis, tels que l'extrapolation de la loi de Hellmann pour l'éolien et la méthode de comptage *Rainflow* pour la dégradation des batteries, offrent au modèle une plus grande précision.

L'étude comparative finale entre la France et le Royaume-Uni montre la conclusion principale de notre projet : il n'existe pas de micro-réseau optimal universel. L'intensité carbone du réseau national de secours et les conditions météorologiques locales changent directement la logique d'investissement et de gestion. Un système conçu pour être écologique en France deviendrait financièrement insoutenable à Londres, et inversement.

Code Source et Reproductibilité
L'intégralité du code source du projet (incluant les scripts de traitement de données géopolitiques, l'optimisation multi-objectif NSGA-II et la génération des Fronts de Pareto) est accessible publiquement sur GitHub à l'adresse suivante :
`https://github.com/m4rt1nschulz/Optimization-of-Microgrid-for-Data-Centers`

Références

[1] Julius Irion, *et al.*, "Optimizing Microgrid Composition for Sustainable Data Centers", *arXiv preprint*, 2025. Disponible sur : `https://arxiv.org/abs/2508.04284`

[2] Philipp Wiesner, *et al.*, "Vessim : A Testbed for Carbon-Aware Applications and Systems.", *ACM SIGENERGY Energy Informatics Review 4 (5).*, 2024. Disponible sur : `https://dl.acm.org/doi/pdf/10.1145/3727200.3727210`

[3] J. Blank et K. Deb, "Pymoo : Multi-Objective Optimization in Python". Disponible sur : `https://pymoo.org/`

[4] National Renewable Energy Laboratory (NREL), *System Advisor Model (SAM).* Disponible sur : `https://sam.nrel.gov/`

[5] RTE France, *éCO2mix - Données de production et de consommation électriques en temps réel*, Données 2022. Disponible sur : `https://www.rte-france.com/eco2mix`

[6] National Energy System Operator (NESO), *Historic Carbon Intensity Data*, Données 2022. Disponible sur : `https://www.neso.energy/`

[7] Fraunhofer ISE, *Energy-Charts : Prix de l'électricité sur le marché journalier (Day-Ahead)*, Données 2022. Disponible sur : `https://energy-charts.info/`

[8] ADEME, *Base Empreinte : Facteurs d'émission pour l'analyse de cycle de vie*, Ministère de la Transition Écologique. (Source des valeurs de CAPEX CO2 : 200 kg/kW éolien, 600 kg/kWp solaire, 100 kg/kWh batterie).

[9] Ministère de l'Économie, des Finances et de la Souveraineté industrielle, *Composante Carbone (Taxe Carbone).*

Annexe : Description détaillée des fichiers du simulateur Vessim

La description complète du simulateur Vessim avec la fonction et variables de chaque fichier est disponible dans la Table 3.

Table 3 – Cartographie exhaustive des fonctions et variables de l'architecture Vessim

Fichier	Fonction	Variables + Leviers d'action
Init	Point d'entrée et organisateur. Définit l'architecture en important les composants (Acteurs, Batteries, Contrôleurs).	Aucune variable de réglage direct dans ce fichier.
Broker	Gère la communication API entre la simulation et l'extérieur via un serveur. Collecte les données de séries chronologiques (Prometheus).	— **microgrids** : Configurations et état actuel. — **history** : Historique des données reçues. — **commands** : Actions en attente. — **port** : Port réseau du serveur Broker. — **export_prometheus** : Exportation des métriques. — **storage_prop** : Propriétés de la batterie. — **policy_prop** : Logique de décision/optimisation. — **actor_signal** : Ajustement de puissance des acteurs. — **microgrid_battery_soc** : État de charge (0 à 1). — **microgrid_battery_capacity_wh** : Capacité totale.

Suite à la page suivante...

Table 3 – Suite de la page précédente

Fichier	Fonction	Variables + Leviers d'action
Data	Gestion des datasets (météo/carbone). Responsable du téléchargement, de l'organisation et de l'ajustement (échelle/temps).	— **VESSIM_DATASETS** : Liste des bases de données. — **dataset** : Nom de la base choisie. — **dir_path** : Chemin d'enregistrement. — **params** : Dictionnaire (scale, start_time, etc.). — **scale** : Facteur d'échelle (multiplicateur). — **start_time** : Date/heure de début des données. — **use_forecast** : Activation des prévisions. — **allowed_parameters** / **required_files** : Sécurité. — **actual** / **forecast** : Données réelles et prévues. — **fill_method** : Remplissage des données manquantes. — **_shift(...)** : Décalage temporel des indices. — **index_cols** : Colonnes d'index temporel.
Util	Utilitaires : conversion d'horloge, filtrage de logs et simplification des structures de données.	— **sim_start** : Ancre de l'horloge virtuelle. — **behind_threshold** : Limite de retard tolérée. — **simtime** : Temps écoulé en secondes. — **dt** : Temps converti en format standard. — **record** : Enregistrement de log interne. — **is_warning** / **is_mosaik_log** / **is_user_warning** : Variables de contrôle de l'affichage. — **message_parts** / **is_below_threshold** : Analyse du retard de simulation. — **d** / **parent_key** / **items** / **new_key** : Variables d'organisation et simplification de dictionnaires.

Suite à la page suivante...

Table 3 – Suite de la page précédente

Fichier	Fonction	Variables + Leviers d'action
Actor	Définit les producteurs/consommateurs. Connecte les composants physiques aux signaux de données.	— **name** / **signal** : Identifiant et source de données. — **step_size** : Intervalle de mise à jour. — **now** / **time** / **self.clock** : Gestion du temps. — **self.power** : Puissance instantanée (W). — **self.state** : Résumé (nom, signal, puissance). — **META** / **self.eid** / **sid** : Variables Mosaik. — **time_resolution** / **num** : Paramètres d'instance. — **model** / **model_params** / **sim_params** : Paramètres d'initialisation et de modèle. — **inputs** / **outputs** / **max_advance** : Flux de données co-simulées.
Controller	Moniteur et décideur. Observe l'état, sauvegarde les données et reçoit les commandes API.	— **controllers** : Liste des contrôleurs actifs. — **self.eid** / **sid** : Identification Mosaik. — **export_prometheus** / **broker_port** / **broker_url** : Communication externe. — **outfile** / **self.filepath** / **self.fieldnames** : Gestion des logs CSV (chemin et colonnes). — **step_size** : Pas d'analyse du contrôleur. — **target** / **prop** / **val** : Paramètres des commandes. — **microgrid_states** / **commands** : États du réseau et ordres en attente. — **self.log** : Historique en mémoire. — **mode** / **write_header** : Contrôle de l'écriture CSV.

Suite à la page suivante...

Table 3 – Suite de la page précédente

Fichier	Fonction	Variables + Leviers d'action
Environnement	Gestionnaire de co-simulation. Connecte tous les sous-systèmes et initialise le framework Mosaik.	— **sim_start** / **step_size** : Configuration temporelle. — **COSIM_CONFIG** : Configuration des scripts. — **microgrids** / **controllers** : Liste des instances. — **until** : Temps de fin de simulation. — **rt_factor** : Facteur temps réel (1.0, 0.5 ou None). — **print_progress** : Barre de progression. — **behind_threshold** : Seuil d'alerte de retard. — **SiL check** : Vérification des données en direct.
Microgrid	Nœud électrique central. Calcule le bilan de puissance et gère l'interaction avec le réseau public.	— **step_size** / **name** : Configuration de base. — **actors** / **policy** / **storage** : Composants connectés. — **grid_signals** : Signaux externes (prix/carbone). — **p_delta** : Bilan de puissance (Watts). — **p_grid** : Échange effectif avec le réseau. — **actor_states** / **policy_state** / **storage_state** : États actuels des composants. — **self.actor_entities** / **self.grid_entity** / **self.storage_entity** : Entités internes Mosaik.
Plot	Outils de visualisation interactive via Plotly (traces brutes et tableaux de bord de résultats).	— **trace** / **scale** : Données et échelle du signal. — **default_visible** : Visibilité des courbes. — **title** / **y_axis_title** / **dataset_name** : Étiquetage. — **df** : Données historiques (MemoryLogger). — **microgrid_name** : Nom du réseau cible. — **actor_states.[name].power** / **p_delta** / **storage_state.soc** / **p_grid** : Indicateurs visualisés.

Suite à la page suivante...

Table 3 – Suite de la page précédente

Fichier	Fonction	Variables + Leviers d'action
Policy	Algorithme décisionnel. Gère la distribution de l'énergie (p_delta) entre stockage et réseau.	— **mode** : "grid-connected" ou "islanded". — **charge_power** : Force la puissance de la batterie. — **p_delta** / **duration** : Données d'entrée pour calcul. — **energy_delta** : Quantité d'énergie (Joules/Wh). — **storage** : Référence à l'instance de batterie.
Signal	Gestionnaire des sources de données (statiques, historiques CSV ou API temps réel).	— **value** : Valeur numérique constante. — **actual** / **forecast** : Vérité terrain et prévisions. — **fill_method** : Interpolation des données. — **scale** / **start_time** : Ajustements (via load). — **update_interval** / **timeout** : Paramètres SiL. — **PrometheusSignal (url, query)** : Métriques externes. — **WatttimeSignal** : API Carbone (user, pass, region).
Storage	Simulateur physique des batteries (Simple ou Lithium-ion avancée ClcBattery).	— **capacity** : Capacité totale en Wh. — **initial_soc** / **min_soc** : Limites de charge. — **c_rate** : Taux maximal de charge/décharge. — **number_of_cells** : Redimensionnement du pack (Clc). — **eta_d** / **eta_c** : Inefficacités (pertes). — **alpha_d** / **alpha_c** : Limites de courant. — **power** / **duration** : Paramètres de mise à jour. — **charge_level** / **_soc** : État interne mis à jour.